Dokumentasi Kode — Entry Point Transmitter

`receiverArduino.py`

Dokumen ini menjelaskan berkas `receiverArduino.py`, yaitu **satu-satunya titik masuk (entry point)** sisi *Pengendali Utama* (Raspberry Pi / PC) untuk mengendalikan robot feeder akuatik. Menjalankan file ini akan menjalankan seluruh sistem kendali: GUI, komunikasi radio, dan logging.

> Dokumentasi arsitektur GUI yang lebih mendalam (analisis kebutuhan, perancangan, pengujian) ada pada `Dokumentasi_GUI_Feeder_SDLC.pdf` di folder yang sama. Dokumen ini fokus pada **kode** `receiverArduino.py` dan modul yang dipanggilnya.

---

1. Gambaran Umum

`receiverArduino.py` berperan sebagai **peluncur (launcher)** dengan tiga mode operasi:

Mode	Cara jalan	Kegunaan
GUI layar penuh (default)	`python receiverArduino.py`	Panel kontrol di layar sentuh 7 inci — dipakai operator di lapangan
GUI jendela	`python receiverArduino.py --windowed`	Jendela 800×480 untuk pengembangan & uji di PC
CLI	`python receiverArduino.py --cli`	Mode teks — uji cepat perintah tanpa layar sentuh

Ditambah satu saklar yang berlaku untuk semua mode: `--real`. Tanpa flag itu aplikasi **selalu mulai di mode SIMULASI** (robot virtual) — pilihan yang aman untuk latihan & demo — dan operator memindahkannya ke NRF sungguhan lewat tombol **MODE** di bilah atas GUI, pemilih **MODE KERJA** di halaman SISTEM, atau perintah `MODE real` di CLI.

Filosofi desainnya: **logika inti tidak ditulis di sini**. Berkas ini hanya memilih mode lalu mendelegasikan pekerjaan ke modul lain:

- `feeder_gui.py` — antarmuka grafis (kelas `FeederGUI`).
- `feeder_core.py` — mesin di baliknya: `RobotCommand`, `RadioLink`, `CommandRunner`, path planner, `ActivityLogger`, `MPUMonitor`.

Dengan begitu `receiverArduino.py` tetap ringkas dan mudah dibaca sebagai peta seluruh sistem.

---

2. Struktur Berkas

Tiga fungsi + satu *guard*:

Fungsi	Peran
`run_gui(fullscreen, simulation)`	Membuka panel kontrol grafis
`run_cli(simulation)`	Menjalankan antarmuka teks
`main()`	Memilih mode dari argumen baris perintah
`if __name__ == "__main__"`	Titik eksekusi saat file dijalankan langsung

---

3. Pemilihan Mode — `main()`

Sangat sederhana. Bila argumen `--cli` atau `-c` ada, jalankan mode teks; jika tidak, buka GUI — layar penuh kecuali diberi `--windowed`:

```
def main():
    # Default SIMULASI; --real meminta NRF sungguhan sejak awal.
    simulation = "--real" not in sys.argv
    if "--cli" in sys.argv or "-c" in sys.argv:
        run_cli(simulation=simulation)
    else:
        run_gui(fullscreen="--windowed" not in sys.argv, simulation=simulation)
```

---

4. Mode GUI — `run_gui()`

Mengimpor `FeederGUI` dari `feeder_gui.py`, lalu memanggil `FeederGUI(fullscreen=...).mainloop()`.

Bagian penting adalah **penanganan** `ImportError`: bila pustaka GUI (mis. `customtkinter`) belum terpasang, program tidak *crash* dengan *traceback* membingungkan, melainkan mencetak petunjuk pemasangan yang jelas lalu keluar rapi:

```
try:
    from feeder_gui import FeederGUI
except ImportError as e:
    print("Gagal memuat GUI:", e)
    print("    pip install customtkinter          (wajib utk GUI)")
    print("    pip install pyrf24                 (utk NRF nyata di Pi)")
    print("    python receiverArduino.py --cli     (mode teks)")
    sys.exit(1)
```

> **Catatan pemasangan (penting di Windows/PC dengan banyak Python):** pesan `customtkinter belum terpasang` muncul bila paket terpasang di interpreter Python yang **berbeda** dari yang menjalankan skrip. Pastikan memasang ke interpreter yang sama, mis. `& "C:\path\ke\python.exe" -m pip install customtkinter`. Di PC (tanpa NRF), `pyrf24` boleh dilewati — GUI memang mulai di **mode simulasi**.

Tentang layar penuh. GUI mengambil resolusi layar perangkat dan berjalan tanpa bilah judul, dirancang untuk layar sentuh 7 inci (800 × 480 px). Tombol **Escape** atau **F11** mengembalikannya ke mode jendela — berguna saat pengembangan di PC. Untuk langsung memulai dalam jendela, pakai `--windowed`.

Tentang mode kerja. GUI dibuka dengan `FeederGUI(fullscreen=..., simulation=...)`. Nilai bawaan `simulation=True` berarti panel **tidak pernah menyentuh radio nyata sampai operator memintanya** — perilaku yang disengaja supaya demo di ruang sidang atau latihan di darat tidak membuat robot bergerak. Menekan tombol **MODE** memanggil `RadioLink.set_mode()`, yang melepas radio lama, membersihkan sisa sinyal `ON`/`FIN` serta telemetri, lalu memasang lawan bicara baru. Bila mode **REAL** diminta padahal `pyrf24`/modul NRF tidak ada, koneksi kembali ke simulasi dan kegagalannya **dilaporkan lewat dialog & log** — bukan diam-diam. Perpindahan mode ditolak selagi ada perintah berjalan, supaya robot tidak ditinggalkan bergerak tanpa pengendali.

---

5. Mode CLI — `run_cli()`

Berbeda dengan versi sebelumnya yang menulis ulang seluruh logika radio, mode teks sekarang **memakai komponen yang sama persis dengan GUI**: `RadioLink` untuk radio dan `CommandRunner` untuk protokol Request-Response. Jadi tidak ada lagi dua implementasi protokol yang bisa saling menyimpang.

5.1 Inisialisasi

```
link = RadioLink()
link.begin(force_sim=simulation)   # default: robot virtual
logger = ActivityLogger(on_new=lambda ev: print(f"  [{ev.category}] {ev.message}"))
runner = CommandRunner(link, logger=logger)
```

```
mpu    = MPUMonitor(link, logger=logger)
```

`ActivityLogger` diberi *callback* yang langsung mencetak tiap kejadian, sehingga operator melihat siklus protokol berjalan secara langsung. Satu *thread* latar mengirim `PING` berkala dan memanggil `mpu.poll()` — sama seperti di GUI — sehingga sudut MPU tetap mengalir walau tidak ada perintah berjalan.

> Parameter radio (**alamat** `00001`**, channel** `0x76`**, data rate** `250KBPS`**, PA** `LOW`) diatur di dalam `RadioLink` dan **harus identik** dengan firmware robot `Finalnew.ino`; jika berbeda, kedua modul tidak akan terhubung. Perhatikan pin CE/CSN di Raspberry (`24, 0`) berbeda dari ESP32 (`4, 5`) karena beda papan.

5.2 Perintah yang tersedia

Ketikan	Aksi
`MAJU <0.1-50>`	Maju sekian **meter** — dipecah otomatis menjadi beberapa paket `FW`
`FW <10-200>`	Satu paket maju, satuan sentimeter (setara paket protokol)
`CW <1-180>`	Putar kanan sekian derajat
`CCW <1-180>`	Putar kiri sekian derajat
`MISI <p> <l>`	Susun sapuan zigzag tambak p × l meter, lalu jalankan seluruh antreannya
`MODE <real|sim>`	Pindah lawan bicara radio
`MPU <on|off>`	Nyalakan/matikan pencatatan sudut MPU dari telemetri NRF
`SIMPAN <on|off>`	Nyalakan/matikan auto-simpan log ke berkas TXT
`STOP`	Berhenti darurat
`KELUAR`	Kirim STOP lalu keluar

5.3 Alur eksekusi

Perintah tunggal divalidasi dulu terhadap tabel rentang, baru dijalankan:

```
cmd = RobotCommand(verb, int(parts[1]))
valid, msg = cmd.validate()
if not valid:
    print(msg)                    # mis. "CW hanya menerima 1-180 derajat (diberi 999)"
    continue
runner.run([cmd], label="MANUAL")
```

Untuk `MISI`, tambak diterjemahkan menjadi antrean perintah lebih dulu:

```
plan  = plan_coverage(panjang, lebar)
queue = plan_command_queue(plan)       # -> [FW:200, FW:200, ..., CW:90, ...]
runner.run(queue, label="MISI")
```

Contoh keluaran nyata:

```
perintah> FW 150
  [KIRIM] [FW:150] percobaan 1 - ACK
1 perintah dijalankan
  [GERAK] ON FW diterima - Maju 150 cm
  [SELESAI] FIN FW:150 - Maju 150 cm tuntas
  [MANUAL] SELESAI - 1 perintah tereksekusi
  -> SELESAI
```

---

6. Konsistensi Protokol dengan Firmware

Paket yang dikirim harus dipahami `Finalnew.ino`. Protokolnya mengikuti *Dokumen Spesifikasi Teknis* bab Skema Komunikasi:

Paket	Arah	Ditangani firmware
`FW:<10-200>`	Pi → Robot	Maju lurus sekian sentimeter
`CW:<1-180>`	Pi → Robot	Putar searah jarum jam (ke kanan)
`CCW:<1-180>`	Pi → Robot	Putar berlawanan jarum jam (ke kiri)
`STOP:0`	Pi → Robot	Hentikan motor — berhenti darurat
`PING`	Pi → Robot	Segarkan *heartbeat*, cegah *failsafe*
`ON <Moves>`	Robot → Pi	Perintah sedang dijalankan (tiap 1 detik)
`FIN <Moves>:<Dist>`	Robot → Pi	Perintah selesai — izin lanjut ke perintah berikutnya
`R=..;P=..;Y=..`	Robot → Pi	Telemetri sudut MPU

Pola Request-Response:

```
Pi  ->  FW:200
Pi  <-  ON FW              (< 1 detik, kalau tidak -> kirim ulang, maks 5x)
Pi  <-  ON FW              (berulang tiap 1 detik selama bergerak)
Pi  <-  FIN FW:200         (selesai -> lanjut perintah berikutnya)
```

Bila `ON` tidak datang dalam 1 detik, `CommandRunner` menganggap perintah gagal dan mengirimnya ulang. Setelah 5 percobaan tanpa jawaban, antrean dihentikan dan `STOP:0` dikirim sebagai pengaman.

---

7. Peta Modul (Arsitektur)

`receiverArduino.py` adalah lapisan tertipis; kekuatan sistem ada di modul yang dipanggilnya:

Berkas	Isi utama	Tanggung jawab
`receiverArduino.py`	`run_gui`, `run_cli`, `main`	Entry point & pemilihan mode
`feeder_gui.py`	kelas `FeederGUI`	Presentasi: enam halaman bergaya ATM, papan angka, preview jalur, gauge, LED, tampilan log
`feeder_core.py`	`RobotCommand`	Satu paket `[Moves]:[Dist]` + validasi rentang sesuai tabel perintah
	`RadioLink`	Abstraksi NRF24 (`pyrf24`) + `MockRadio` (simulasi); merutekan `ON`/`FIN`/telemetri
	`plan_coverage()` / `plan_command_queue()`	Tambak persegi + titik awal + lebar lintasan → segmen "maju/belok" → antrean perintah FW/CW/CCW
	`CommandRunner`	Pelaksana Request-Response: kirim, tunggu `ON` (kirim ulang bila lewat 1 detik), tunggu `FIN`, lanjut
	`ActivityLogger`	Pencatatan aktivitas berstempel waktu, ekspor CSV/TXT, dan **auto-simpan** tiap baris ke berkas TXT
	`MPUMonitor`	Ambil sudut MPU (roll/pitch/yaw) dari telemetri NRF di latar belakang + catat ke log bila diminta

Perbedaan GUI vs CLI. Keduanya memakai protokol dan komponen inti yang sama. GUI menambahkan: papan angka layar sentuh, input titik awal / lebar lintasan / kecepatan, preview jalur pada kanvas, gauge telemetri, panel sensor MPU, dan ekspor log. CLI hanya mengirim perintah dan mencetak kejadian.

Komunikasi 2 arah half-duplex ("gantian"). Satu modul NRF tidak bisa kirim dan terima bersamaan, jadi kedua sisi bergantian seperti radio HT. `RadioLink.pump()` membuka jendela dengar singkat lalu kembali ke mode TX; semua paket yang masuk dirutekan menurut awalannya — `ON`, `FIN`, atau telemetri MPU. Akses radio dilindungi *lock* agar thread heartbeat dan thread runner tidak menulis bersamaan.

Catatan thread (penting). `CommandRunner` berjalan di thread latar, sedangkan Tkinter hanya boleh disentuh dari thread utama. Karena itu runner **tidak** diberi *callback* ke GUI — tampilan justru **menarik** (polling) status dan log tiap 120 ms. Memanggil `after()` dari thread latar pernah membuat thread runner terblokir menunggu Tcl sehingga sinyal `ON` tidak pernah terbaca.

---

8. Cara Menjalankan

```
cd "Master Code/Transmitter"
python receiverArduino.py               # GUI layar penuh (layar sentuh 7")
python receiverArduino.py --windowed    # GUI jendela 800x480 (uji di PC)
python receiverArduino.py --real        # langsung memakai NRF sungguhan
python receiverArduino.py --cli         # mode teks
```

Dependensi (lihat `requirements.txt`):

Paket	Untuk	Wajib di
`customtkinter`	framework GUI	GUI (PC & Pi)
`pyrf24`	driver NRF24L01	**hanya Raspberry Pi** (dilewati di Windows)

Di **PC** cukup `pip install customtkinter` — mode SIMULASI tidak butuh apa pun selain itu. Di **Raspberry Pi** tambahkan `pyrf24` untuk kendali radio nyata. `pyserial` **tidak lagi dibutuhkan**: sudut MPU ikut telemetri NRF, bukan kabel USB. Selalu pasang ke interpreter Python yang sama dengan yang menjalankan skrip.

---

9. Ringkasan

`receiverArduino.py` sengaja dijaga **minimal dan jelas**: satu entry point, tiga mode, penanganan error yang ramah, dan pendelegasian bersih ke `feeder_gui.py`/`feeder_core.py`. Sejak mode CLI ikut memakai `RadioLink` dan `CommandRunner`, tidak ada lagi duplikasi implementasi protokol — perbaikan pada protokol otomatis berlaku di kedua mode.